Cpts 317. 11/30.

Today: there is a language that is r.e. but not recursive.

// there is a problem that can't be solved by algs.
// a language.

// Undecidability.

We define

$$\mathcal{L}_u = \{\langle M, w\rangle : \text{TM } M \text{ says yes on } w\}$$

// Intuitively,
// M ——— a C-program, a string.
// w ——— input to the program, a string.
// $\langle M, w\rangle$ ——— a string, concatenated by the above two strings (sep. by a delimiter)

Thm. L_u is r.e. That is, there is a TM M_u to accept L_u.

Proof. We build M_u:

input $\langle M, w \rangle$

decode the input $\langle M, w \rangle$ to obtain TM M and an input word w

Run M on w;

if M says yes on w, then ret yes;

Clearly, M_u accepts L_u.

// M_u is called universal TM.

// Q.E.D.

Russell's Paradox:

only shaves for those
who don't shave for themselves.

shaves for ?

a string $\xrightarrow{\text{encoded into}}$ a number.

"1234" $\longrightarrow$ $2^1 \cdot 3^2 \cdot 5^3 \cdot 7^4$

primes.

Gödel numbering.

Let M be a TM. M is also a string.

(Every C-program is a string).

$\Rightarrow$ M can be uniquely encoded into a number i.

$\Rightarrow$ We use M_i to denote the i-th TM (whose number encoding is i).

Define $L_{self} := \{ i : i \in L(M_i) \}$.

// $\underline{P}$, a c-program, runs on $\underline{P}$, and then if
// $\underline{P}$ exits nicely, we put $\underline{P}$ into L_{self}.

Define $L_{\neg self} := \{ i : i \notin L(M_i) \}$.

//
// P runs on $\underline{P}$ and $\underline{P}$ doesn't exit nicely.
//
//

Thm. L_{self} is r.e.

Proof. easy. I can build a TM M:

input i

construct M_i and Run M_i on i.

if M_i says yes, set yes.

Thm. $L_{\neg self}$ is not r.e.

Proof. Suppose that $L_{\neg self}$ is r.e. Then, there is a TM M_i, for some i, to accept $L_{\neg self}$; i.e.,

$L_{\neg self} = L(M_i)$.

$i \in L_{\neg self}$ iff $i \notin L(M_i)$

iff $i \notin L_{\neg self}$. a contradiction

$\boxtimes$

Thm. L_u is not recursive.

Proof. (Sketch). Assum that L_u is recursive; Then we have an alg M_u to recog. L_u:

$\forall M, w$

M_u says yes on $\langle M, w \rangle$ iff

M says yes on w;

M_u says no on $\langle M, w \rangle$ iff

M ~~says no~~ does't say yes on w.

From this, we will construct a TM $M_{\neg self}$ to accept $L_{\neg self}$:

$M_{\neg self}$: input i

Run M_u on $\langle M_i, w_i \rangle$

if M_u says ~~yes~~ no, ret yes.

This is a contradiction, since $L_{\neg self}$ is not r.e.

($L_{\neg self}$ can not be accepted by any TM).

So far,

$L_u = \{\langle M, w\rangle : M \text{ says yes on } w\}$ is not recursive.

That is, there is no alg that can solve the following problem:

Given: a TM M, a word w

Quest: Does M say yes on w?

Such problems that we don't have algs to solve are called "Undecidable problems".

Real world undecidable problems:

Given: a C-program P, input x

Ask: P exists normally on input x?

Given: a program P with no input

Ask: P halts?

Halting Problem is undecidable.